김승민

자바 백엔드 개발자

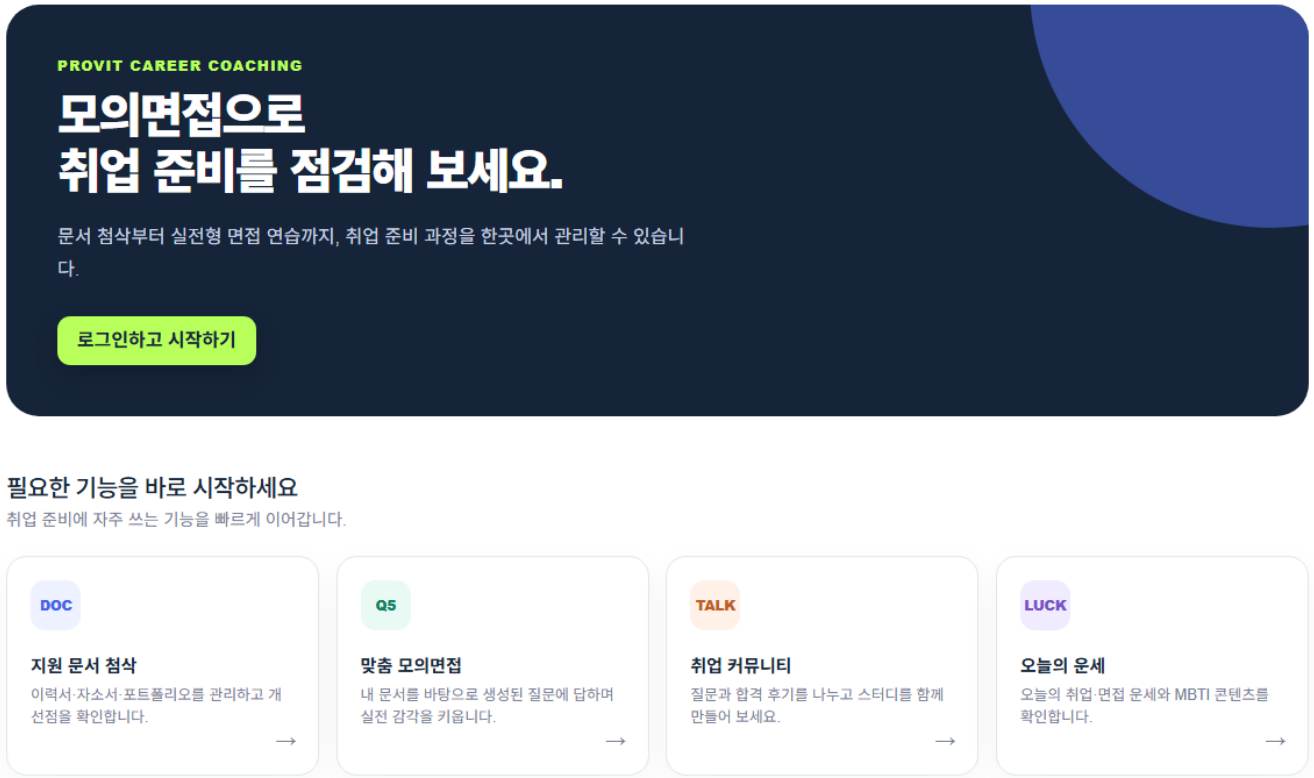
거래 데이터를 자산 정보로,
취업 서류를 AI 서비스로 연결한 경험

frade

거래기록과 자산 포트폴리오

provit

취업문서 / AI 첨삭 / AI 모의면접



peppyksm@gmail.com

github.com/peppyksm

Profile & Skills

프로젝트에서 구현한 기능을 기준으로 기술 경험을 정리했습니다.

기능 구현과 예외 처리까지

김승민

provit 4인 팀의 팀장
프로젝트 총괄 및 일정 관리
취업문서·AI 기능 전체 담당

01 **Java / Spring MVC**
서비스 로직, REST API, 입력 검증, 트랜잭션

02 **MyBatis / Oracle**
JOIN 조회, 복합 데이터 저장, 문자 길이 계약

03 **OpenAI API**
생성 흐름 구성, 응답 검증, 실패와 재전송 처리

04 **React / JSP**
입력 화면, API 연동, 결과·기록 표시

팀장 역할과 프로젝트 일정은 제공된 provit 결과발표자료를 기준으로 정리했습니다.

Project Overview

두 프로젝트에서 담당한 기능과 개발 환경

01 frade

모의 주식거래 + 커뮤니티 서비스

담당 범위

거래기록 저장·전체 조회
최근 거래기록
보유종목·자산 포트폴리오

Java 17 / Spring MVC 5.3.37
MyBatis / Oracle / JSP / Chart.js

02 provit

AI 기반 취업 준비 플랫폼

담당 범위

취업문서 작성·조회·수정·삭제
AI 통합 첨삭
AI 모의면접 전체

2026.09.14 ~ 2026.10.08 / 4인 팀
Java 17 / Spring MVC / React 19 / Oracle 19c

provit 일정: 기획·설계 09.14~17, 개발 09.18~10.07, 통합·테스트 10.06~08

거래기록 조회와 저장

전체 거래내역과 마이페이지에 필요한 최근 기록을 목적에 맞게 조회했습니다.

사용자별 거래 이력

거래 종목, 가격, 수량을 저장하고
사용자 번호로 조회 범위를 제한합니다.

전체·기간별 조회

전체 / 오늘 / 최근 일주일 / 최근 한 달
거래일시 내림차순으로 조회합니다.

마이페이지 최근 25건

종목 테이블과 JOIN해 종목명을 함께 전달합니다.
마이페이지는 최근 25건 조회를 사용합니다.

최근 거래기록 조회 SQL

```
SELECT * FROM (
  SELECT s.stock_name, h.trade_price,
         h.trade_cnt, h.trade_date
  FROM t_history h
  JOIN t_stock s
    ON s.stock_code = h.stock_code
  WHERE h.user_num = #{userNum}
  ORDER BY h.trade_date DESC
) WHERE ROWNUM <= 25
```

구현 근거: HistoryDAO / HistoryDAOImpl / history_mapper.xml. 기간별 조건은 DAO·SQL 구현 기준입니다.

보유종목과 자산 포트폴리오

현재가, 매입원가, 예치금을 묶어 투자 상태를 확인할 수 있게 구성했습니다.

계산 항목	계산 규칙
평가금액	현재가 × 보유수량
종목 손익	평가금액 - 매입원가
종목 수익률	손익 ÷ 매입원가 × 100
총자산	종목 평가금액 합계 + 예치금
자산 비중	종목 평가금액 ÷ 총자산 × 100

화면용 조회 모델 구성

PortfolioInfoDTO에 종목별 계산값을 담고,
MyPagePortfolioDTO로 자산 요약·기록·차트 데이터를 묶어 전달합니다.

계산 경계값 처리

보유수량이 0보다 큰 종목을 조회합니다.
비율 계산의 분모가 0이면 0을 반환합니다.

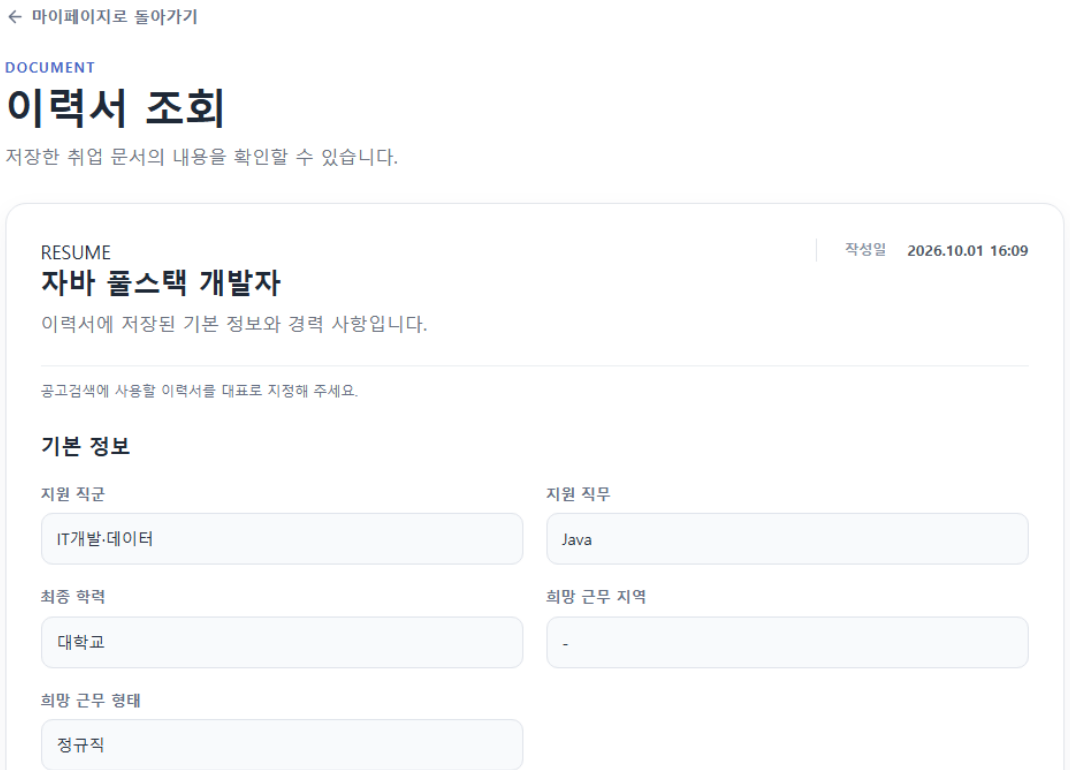
시세 변화 반영

기존 SSE chart-tick을 받아 손익과 자산 비중,
차트를 갱신합니다. 페이지 이탈 시 연결을 닫습니다.

구현 근거: PortfolioServiceImpl.java / portfolio_mapper.xml / user/mypage.jsp

취업문서와 AI 모의면접

서류 관리부터 첨삭, 면접 연습, 완료 기록 조회까지 이어지는 서비스를 개발했습니다.



내가 맡은 기능

취업문서 작성·조회·수정·삭제
AI 통합 첨삭과 첨삭 기록
AI 모의면접의 시작·진행·평가·저장

프로젝트에서의 역할

4인 팀의 팀장으로 프로젝트 총괄과 일정 관리를 맡았습니다. 담당 기능의 화면·API·DB 흐름을 연결했습니다.

Spring MVC 5.3.39 / Tomcat 9 / Java 17
React 19 / MyBatis / Oracle 19c / OpenAI API

왼쪽: 제공된 이력서 조회 화면. 화면 속 이력서 데이터는 서비스 예시이며 지원자의 경력 정보가 아닙니다.

취업문서 작성과 파일 관리

이력서·자기소개서·포트폴리오를 저장하고, 본인의 문서를 AI 입력으로 사용할 수 있게 했습니다.



복합 이력서 저장

이력서 본문과 학력·경력·자격증 목록을
한 트랜잭션으로 저장합니다.
필수값·날짜·직무 코드를 서버에서 검증합니다.

파일 접근과 검증

문서 소유권을 확인하고 PDF의 용량·확장자·시그니처를 검사합니다. UUID 파일명으로 공개 경로 밖에 저장합니다.

DB와 파일의 생명주기

DB 롤백 시 업로드한 새 파일을 정리하고,
문서 삭제는 DB 커밋 후 파일에 반영합니다.

화면: 최종 화면설계 자료. 코드: DocumentServiceImpl.java / LocalPortfolioFileStorage.java

AI 통합 첨삭

선택한 서류 묶음을 한 번에 검토하고, 수정 제안과 원문 근거를 저장합니다.



한 번의 생성 요청

이력서·자기소개서와 선택한 PDF, 첨삭 기준을
함께 전달합니다. 정상 실행은 생성 호출 1회입니다.

응답 검증

JSON 구조·길이·필수 항목을 검증합니다.
텍스트 문서의 인용문이 실제 입력에 있는지 확인하고
PDF의 문서 종류·페이지 범위를 검사합니다.

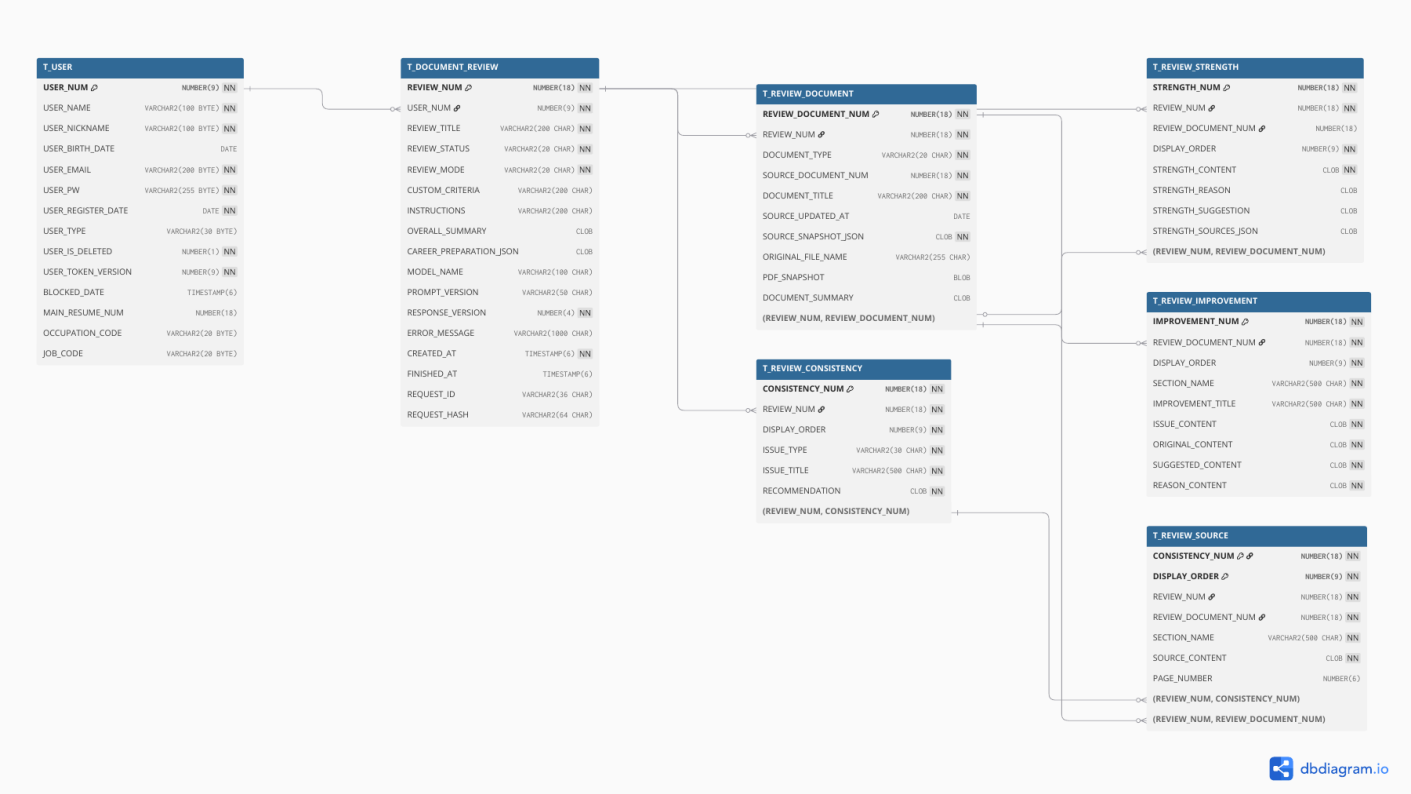
결과와 기록 조회

종합 요약, 문서별 강점·수정 제안,
서류 간 일관성 문제와 취업 준비 추천을 저장합니다.

왼쪽은 화면 구성용 첨삭 결과 예시입니다. 실제 선택 서류에 대한 분석 결과나 성능 지표로 제시하지 않습니다.

중복 실행 방지와 결과 저장

외부 AI 호출과 DB 트랜잭션을 분리하고, 재전송 시 기존 요청을 재사용합니다.



왼쪽: 제공된 첨삭결과 ERD. 코드: DocumentReviewServiceImpl.java / document_review_mapper.xml

requestId + 입력 해시

사용자별 요청 ID와 SHA-256 입력 해시를 확인합니다. 같은 입력은 기존 결과를 반환하고, 다른 입력은 409로 거절합니다.

짧은 DB 트랜잭션

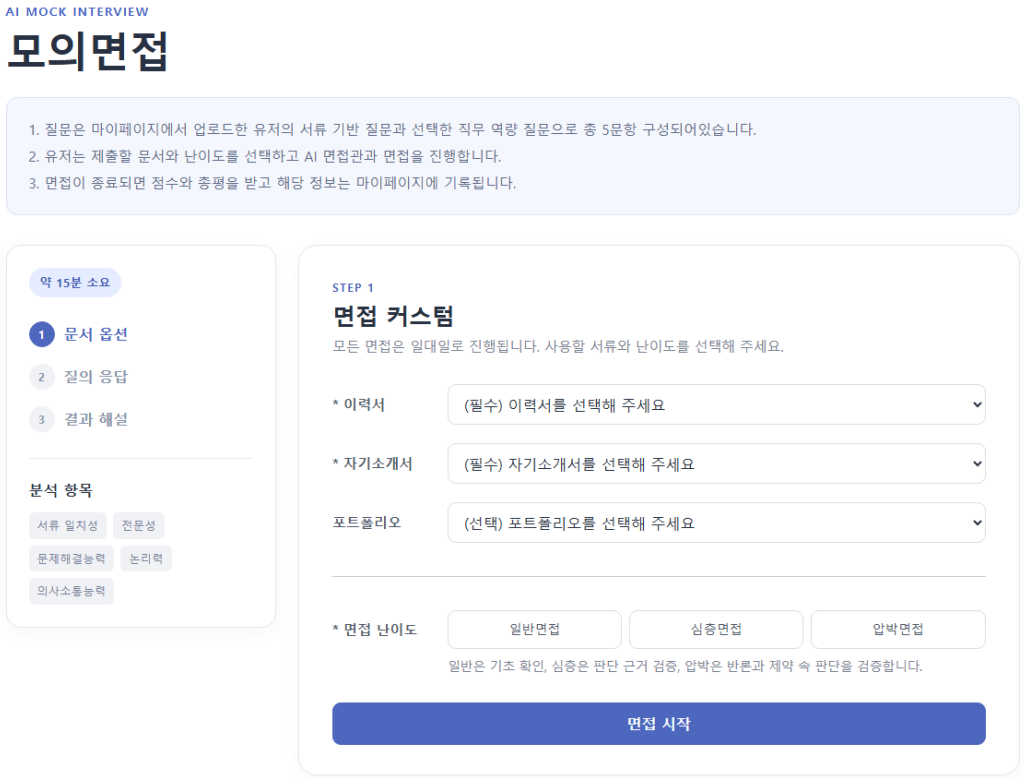
PROCESSING 요청과 원문을 먼저 저장합니다. AI 호출 후 별도 트랜잭션에서 결과와 완료 상태를 저장합니다.

원문 보관본

서류 수정·삭제 이후에도 첨삭 시점의 원문과 결과를 함께 조회할 수 있게 구성했습니다.

5문항, 4번의 생성 호출

서류 기반 질문, 문제해결 질문, 꼬리질문, 최종 평가를 단계별로 구성했습니다.



01 **면접 시작**
1~3번 질문 일괄 생성

02 **3번 답변 제출**
4번 문제해결 질문 생성

03 **4번 답변 제출**
5번 꼬리질문 생성

04 **5번 답변 제출**
전체 응답·서류로 최종 평가

1·2번 답변에는 기존 질문을 반환합니다. 선택한 공고 요약을 반영하며, 4회는 정상 완료 흐름 기준입니다.

면접 평가와 상태 복원

평가 결과를 저장하고, 중단·재전송·저장 실패 상황에서도 진행 상태를 관리합니다.



5개 평가 항목

서류 일치성·전문성·문제해결능력·논리력·의사소통능력을 각각 0~100점으로 받습니다. 종합 점수는 서버에서 평균으로 계산합니다.

재전송과 저장 실패

세션별 동기화로 답변 순서를 관리합니다.
일시 저장 실패에는 평가를 재사용해 저장만 재시도하고, 기록과 결과를 한 트랜잭션으로 저장합니다.

면접 이어가기

서버 상태와 브라우저 초안을 복원하며 시간은 초기화하지 않습니다. 현재 세션은 서버 메모리에 있어 재시작·인스턴스 변경 시 복원할 수 없습니다.

왼쪽은 제공된 테스트 면접 기록 화면입니다. 화면의 점수는 지원자의 기술 숙련도나 개선 성과를 뜻하지 않습니다.

Oracle의 한글 저장 길이 계약

AI 출력 길이, Java 검증, DB 컬럼의 저장 단위를 함께 확인했습니다.

500 BYTE

한글 250자 = 750 BYTE

프로젝트 기록의 AL32UTF8 환경에서
기존 평가 컬럼의 용량을 초과했습니다.

기존 데이터가 새 제한을 넘으면 ALTER 전에 중단합니다.
한글·혼합문자·경계값과 영구 오류 재제출을 테스트 대상으로
구성했습니다.

수정 접근

InterviewTextLimits로 질문·답변·평가 제한을 공유하고,
DB 컬럼을 명시적인 CHAR 기준으로 정의했습니다.

항목	서버·AI 제한	DB 정의
질문	120자	120 CHAR
답변	1,000자	1,000 CHAR
평가 문구	250자	250 CHAR

기존 DB 적용에는 별도 마이그레이션이 필요합니다.

근거: interview-text-length-review.md / OracleInterviewTextLengthTest / InterviewFourCallFlowTest

김승민

자바 백엔드 개발자

데이터의 의미와 서비스의 상태를
끝까지 확인하는 개발자가 되겠습니다.

다음 개선 방향

면접 세션의 공용 저장소 전환
실제 서류 기반 질문·첨삭 품질 평가
호출 지연과 실패율 측정

Contact Us

EMAIL

peppyksm@gmail.com

GITHUB

github.com/peppyksm

자료: frade·provit 로컬 코드, 3조 최종 발표·화면설계·ERD. 디자인 구성 참고: designkeep.co.kr/53/?bmode=view&idx;=123790921